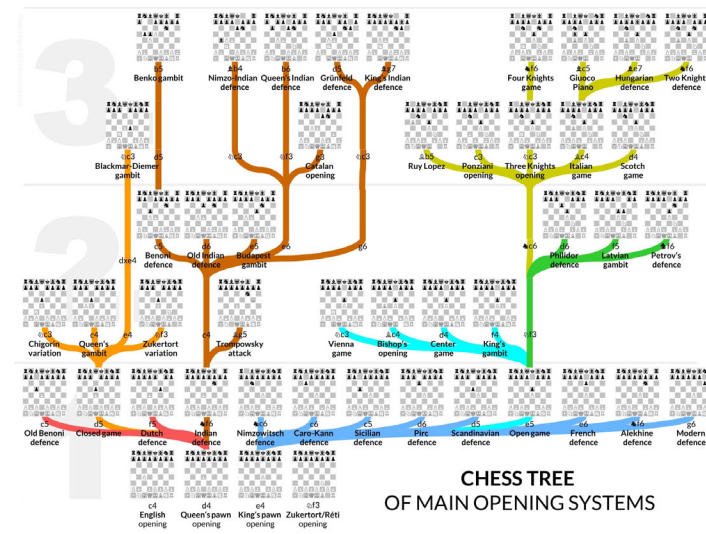


Depth-limited search

Wat als de game tree te groot is ?

- Bijna altijd is de game tree van een spel veel te groot om helemaal recursief door te rekenen.
- We bespreken 3 variaties van minimax die hier een oplossing kunnen bieden



Grootte van een (game) tree

- Om de grootte van een tree aan te duiden:
 - **Branching factor**: het gemiddeld aantal opties op elke node
 - **Depth**: diepte van de boom
- Deze twee getallen bepalen grofweg hoe breed en hoe lang de boom is.
- TicTacToe heeft een branching van ongeveer 4.5, en een maximale depth van 9. Dit is heel klein
- Minimax heeft als algoritme een complexiteit van $O(b^e)$

Limited depth

- In de implementatie van Tic tac Toe zie je al een depth parameter die wordt meegesleept in het algoritme.
- Je kan deze gebruiken om maximaal een aantal levels diep te zoeken

```
if is_maximizing:
    best = -2 # laag genoeg beginnen
    for i in range(3):
        for j in range(3):
            if self.board[i][j] == ' ':
                # X invullen en checken wat er dan precies zou
                # gebeuren, recursieve definitie
                new_move = self.copy()
                new_move.board[i][j] = 'X'
                best = max(best,
                    new_move.minimax(depth + 1, not
                    is_maximizing)) #tegengestelde positie
```

Limited depth

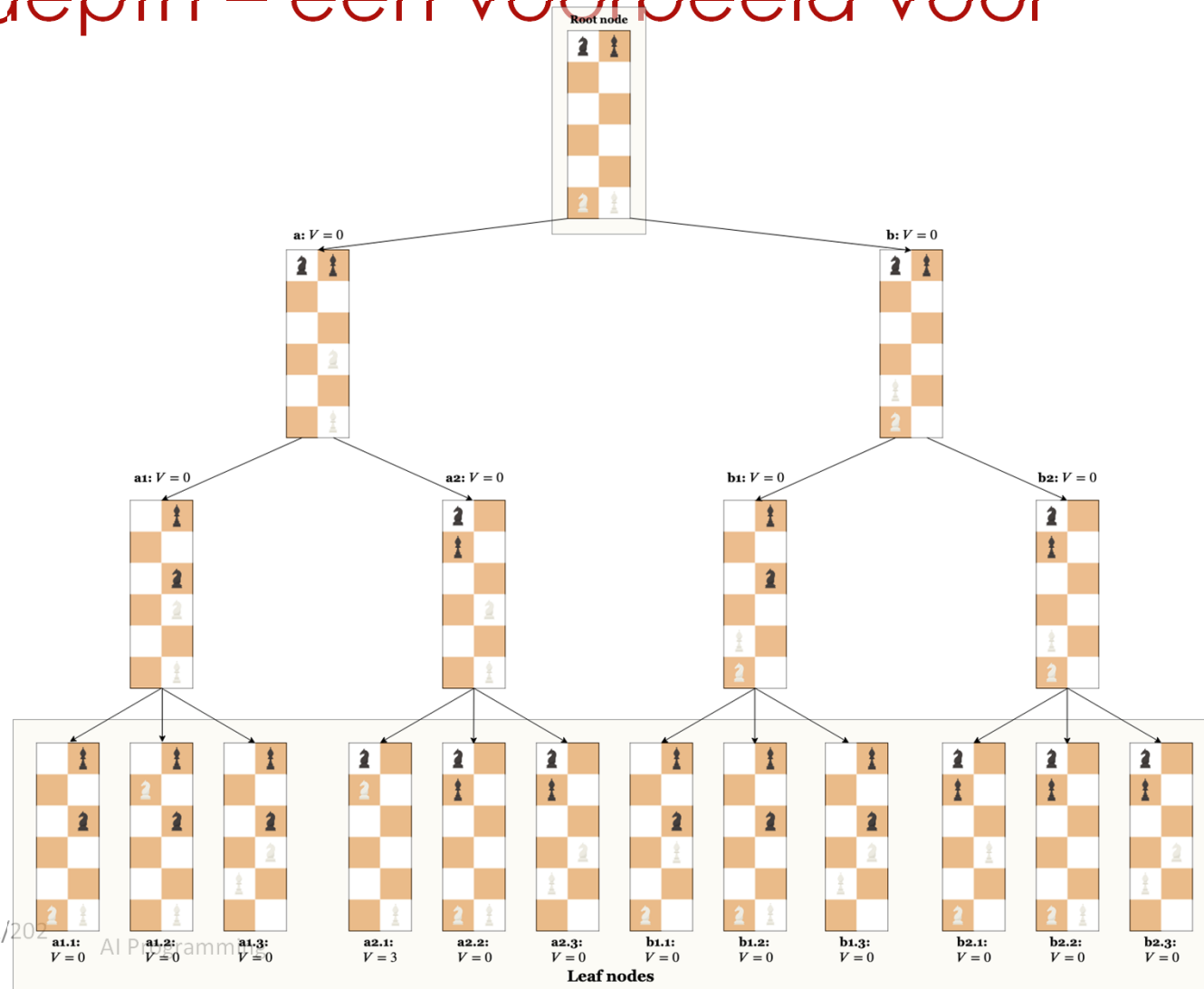
- Je kan je algoritme dan aanpassen om na maximaal 5 levels diep te stoppen
- Maar:
 - Op 5 levels diep heb je mogelijk géén terminale staat
 - Maar je wil daar toch een waarde teruggeven...

```
if is_maximizing:
    best = -2 # laag genoeg beginnen
    for i in range(3):
        for j in range(3):
            if self.board[i][j] == ' ':
                # X invullen en checken wat er dan precies zou
                # gebeuren, recursieve definitie
                new_move = self.copy()
                new_move.board[i][j] = 'X'
                best = max(best,
                    new_move.minimax(depth + 1, not
                    is_maximizing)) #tegengestelde positie
```

Limited depth

- Je kan je algoritme dan aanpassen om na maximaal 5 levels diep te stoppen
- Je hebt dan op elk ogenblik een **inschatting** nodig van hoe goed een huidige bordpositie is
 - Schaken: (aantal stukken van jezelf) – (aantal stukken tegenstander)
 - Risk: hoeveel regio's je bezet (evt met zwaarte leger)
 - Catan en veel andere spellen: hoeveel punten je al hebt
 - ...

Limited depth – een voorbeeld voor schaken



$$\text{King} = \text{Queen} = \text{Rook} = \text{Bishop} = 3$$

a:		+	-		-		= 0
b:		+	-		-		= 0
a1:		+	-		-		= 0
a2:		+	-		-		= 0
b1:		+	-		-		= 0
b2:		+	-		-		= 0
a1.1:		+	-		-		= 0
a1.2:		+	-		-		= 0
a1.3:		+	-		-		= 0
a2.1:		+	-				= 3
a2.2:		+	-		-		= 0
a2.3:		+	-		-		= 0
b1.1:		+	-		-		= 0
b1.2:		+	-		-		= 0
b1.3:		+	-		-		= 0
b2.1:		+	-		-		= 0
b2.2:		+	-		-		= 0
b2.3:		+	-		-		= 0

Uitbreiden 'evaluate' functie

- In onze code hadden wij voorlopig bij 'evaluate' enkel de terminale states:
 - 1 voor winst
 - -1 voor verlies
 - 0 voor gelijkspel
- Een nieuwe evaluatiefunctie moet nu voor *élk* spelbord een schatting kunnen maken

Deze functie noemen we ook wel de '**utility**' functie. Hoe hoger de utility, hoe beter

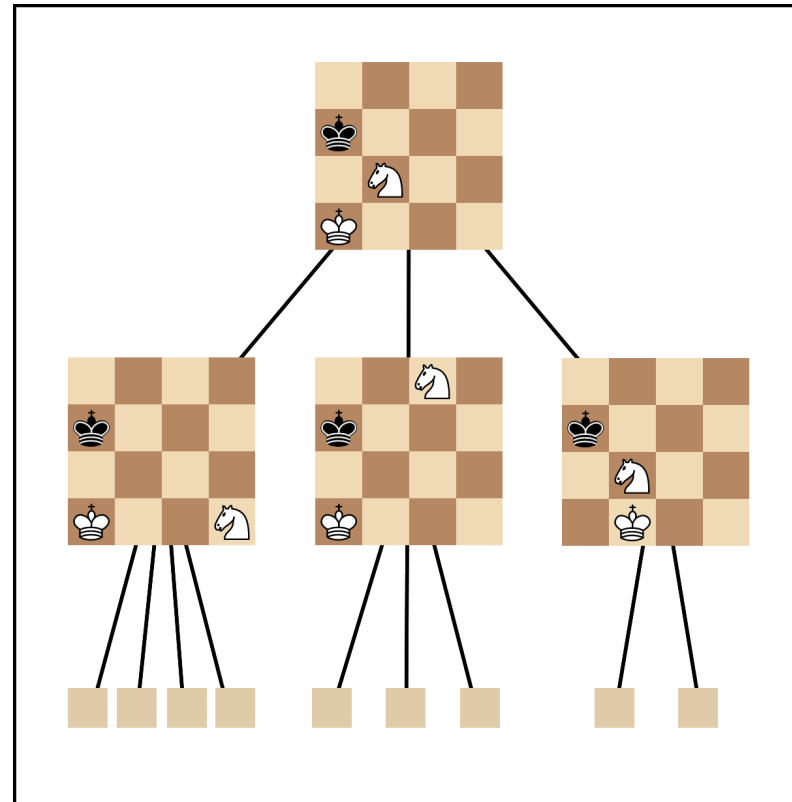
Utility functie voor schaken

```
def evaluate_board(board):  
    # Waardes van stukken  
    piece_values = {  
        'P': 1, 'N': 3, 'B': 3, 'R': 5, 'Q': 9, 'K': 0,  
        'p': -1, 'n': -3, 'b': -3, 'r': -5, 'q': -9, 'k': 0  
    }  
  
    white_score = 0  
    black_score = 0  
  
    # Evaluatie  
    for row in board:  
        for square in row:  
            if square in piece_values:  
                if square.isupper():  
                    white_score += piece_values[square]  
                else:  
                    black_score += piece_values[square]  
  
    return white_score - black_score
```

Utility functie

- Het werken met een utility functie is **nodig** om limited depth minimax te kunnen doen
- Het heeft een belangrijk nadeel: **de oplossing is niet meer perfect**, en sterk afhankelijk van hoe goed de utility function is geïmplementeerd
- Hier ontstaat een afweging:
 - De utility moet een goede waarde kunnen geven van een spelbord
 - Maar het mag ook niet te lang duren om deze te berekenen

Schaakcode (demo)



Limited depth - implementatietip

- In plaats van je algoritme te laten optellen tot een bepaalde waarde, kun je het ook laten aftellen tot 0
- Dit maakt je code makkelijker te herbruiken, omdat je bij de eerste aanroep kunt aangeven wat de maximale diepte is als parameter

```
if is_maximizing:
    best = -2 # laag genoeg beginnen
    for i in range(3):
        for j in range(3):
            if self.board[i][j] == ' ':
                # X invullen en checken wat er dan precies zou
                # gebeuren, recursieve definitie
                new_move = self.copy()
                new_move.board[i][j] = 'X'
                best = max(best,
                    new_move.minimax(depth - 1, not
                    is_maximizing)) #tegengestelde positie
```

Alpha Beta Pruning

Snoeien in de game tree

Alpha Beta pruning

- Er zijn in veel game trees bepaalde shortcuts te nemen
- Door die shortcuts uit te buiten, moeten we bepaalde stukken van de game tree niet onderzoeken.
- Dit kan ons veel tijd besparen!
- We beschouwen als voorbeeld even terug de TicTacToe tree op volgende slides.
- Het algoritme, zoals het is ontworpen, werkt zich depth-first naar beneden

Minimax Algoritme voor Tic Tac Toe

X aan zet

X		
X	O	
O	X	O

0

MAX

O aan zet

X	X	
X	-1	
O	X	O

X		X
X	0	
O	X	O

X		
X	-1	X
O	X	O

MIN

X aan zet

X	X	O
X	-1	
O	X	O

X	X	
X	+1	O
O	X	O

X	O	X
X	0	
O	X	O

X		X
X	+1	O
O	X	O

X	O	
X	0	X
O	X	O

X		O
X	-1	X
O	X	O

MAX

X	X	X
X	+1	O
O	X	O

X	O	X
X	0	X
O	X	O

X	X	X
X	+1	O
O	X	O

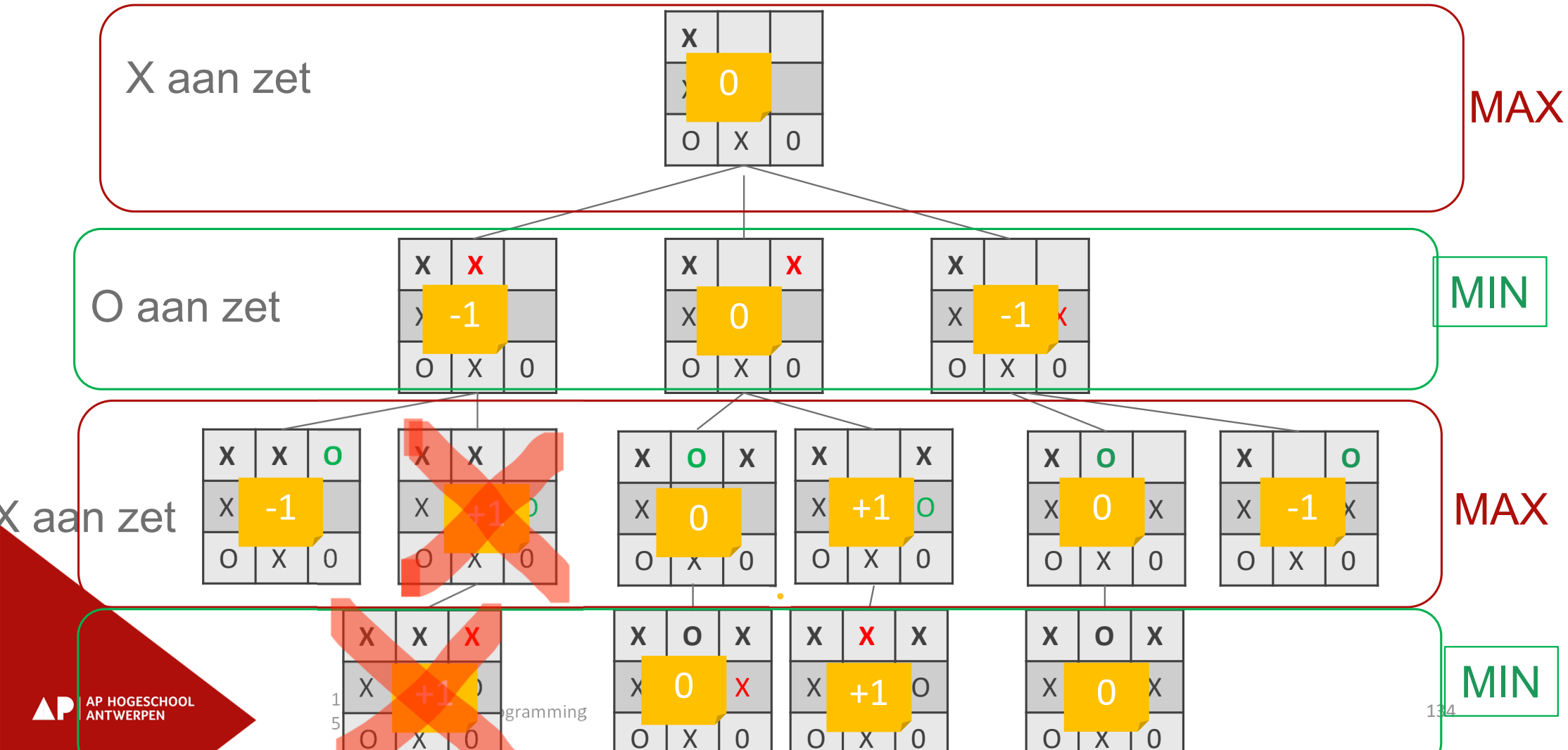
X	O	X
X	0	X
O	X	O

MIN

Minimax Algoritme voor Tic Tac Toe – a-b pruning



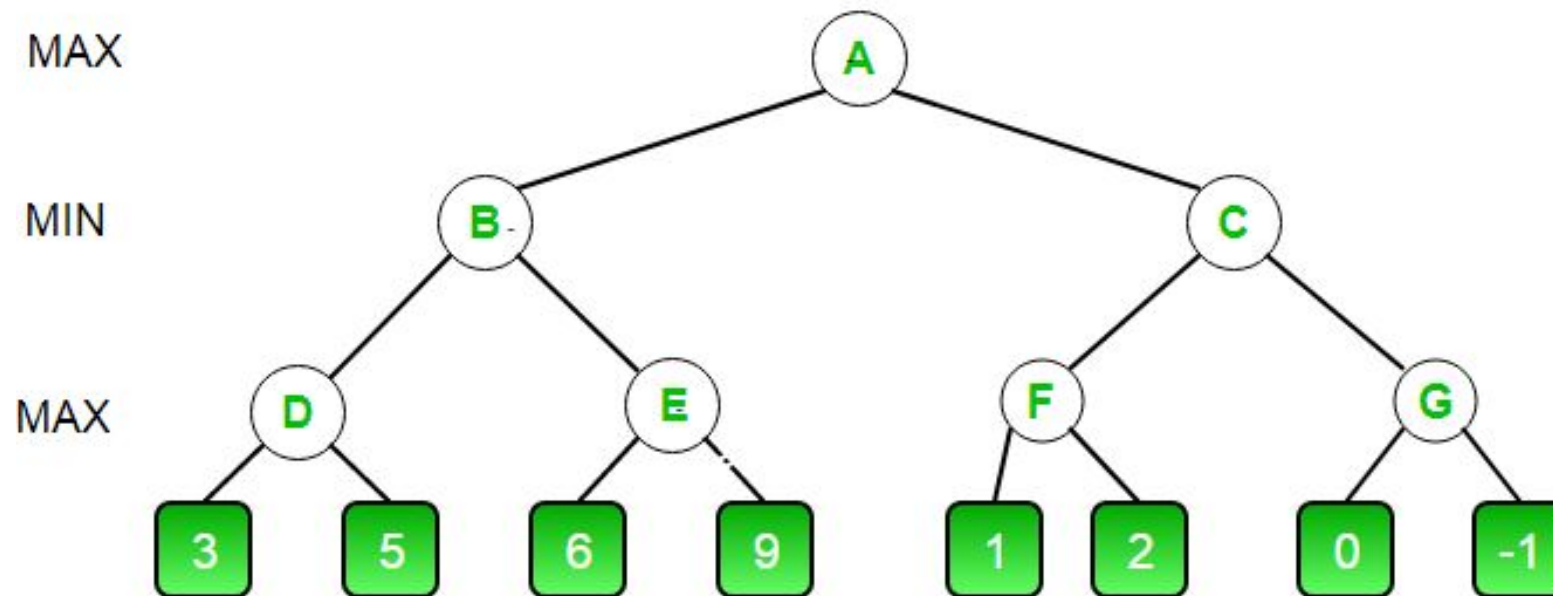
Minimax Algoritme voor Tic Tac Toe – a-b pruning



Werking alpha beta pruning

- Het algoritme houdt 2 waarden bij die het heel de tijd meegeeft aan onderliggende stappen, genaamd **alpha en beta**.
- **Alpha** is initieel $-\infty$
- **Beta** is initieel $+\infty$
- Wanneer we bij een minimizing (*resp maximizing*) fase een waarde tegenkomen die hoger is dan beta (*resp lager dan alpha*), dan moeten we die niet doorlopen
- Dat is in algoritme als **$\beta < \alpha$**
- Bij elke stap updaten we ook **alpha en beta**

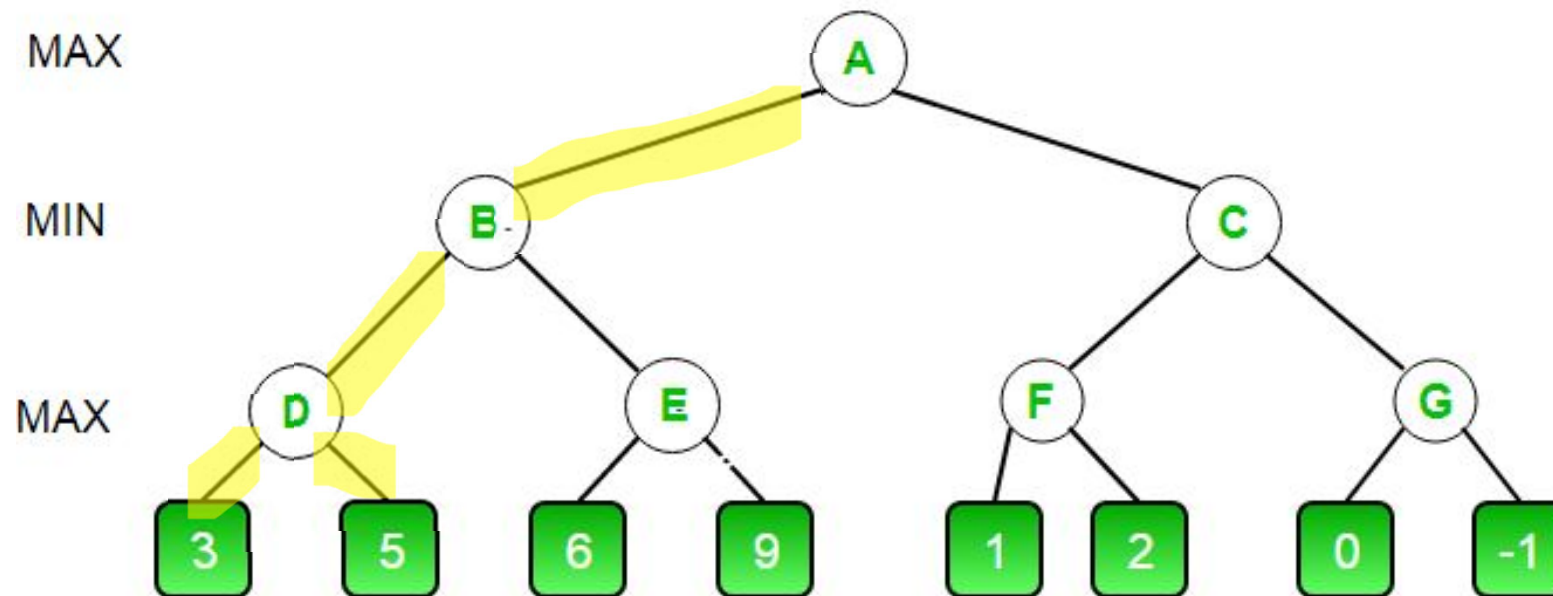
Nog een voorbeeld



bron: <https://www.geeksforgeeks.org/dsa/minimax-algorithm-in-game-theory-set-4-alpha-beta-pruning/>

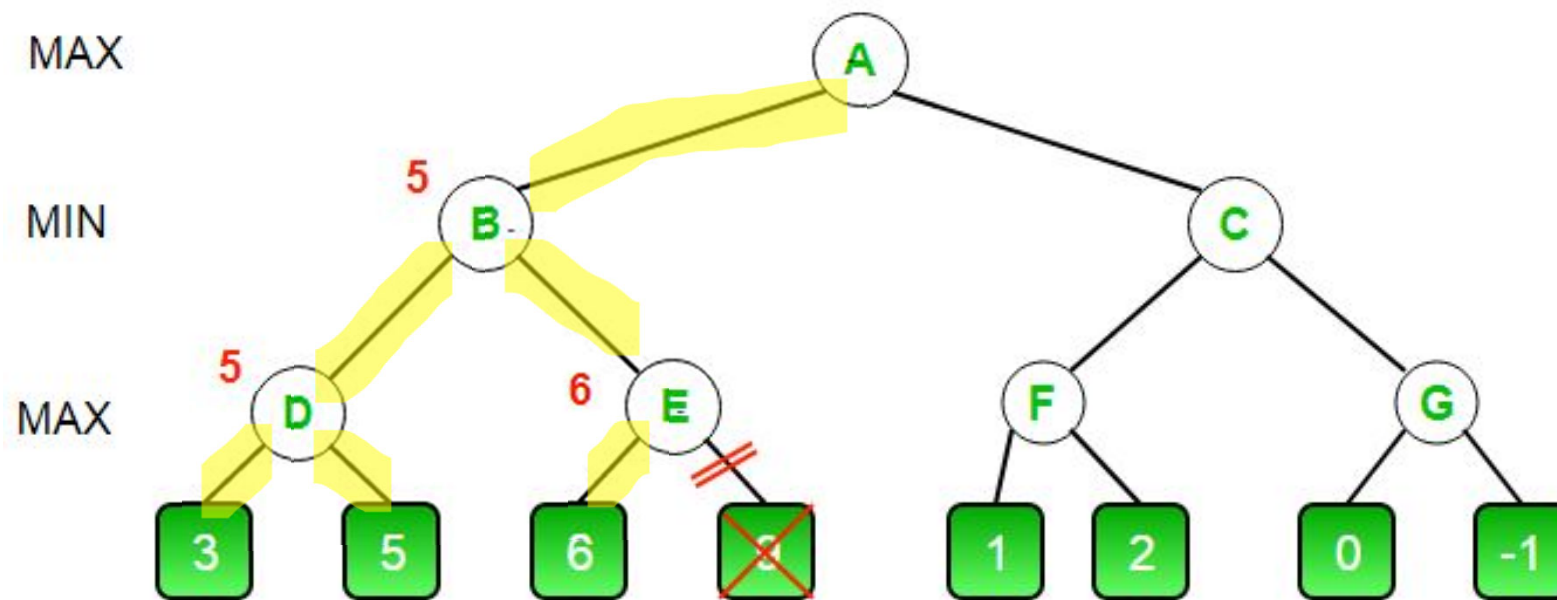
- De initiële aanroep begint bij A. De waarde van alpha is hier -INFINITY en de waarde van beta is +INFINITY. Deze waarden worden doorgegeven aan de volgende nodes in de *tree*. Bij A moet de maximizer kiezen uit de max van B en C, dus A roept eerst B aan.
- Bij B moet de minimizer kiezen uit de min van D en E en roept daarom eerst D aan.
Bij D kijkt hij naar zijn left *child* dat een *leaf node* is. Deze node retourneert een waarde van 3. Nu wordt de waarde van alpha bij D: $\alpha = \max(-\text{INF}, 3)$, wat 3 is.
- Om te beslissen of het de moeite waard is om naar zijn right *child node* te kijken, controleert hij de conditie $\beta \leq \alpha$. Dit is onwaar, omdat $\beta = +\text{INF}$ en $\alpha = 3$. Dus de zoektocht gaat verder.
- D kijkt nu naar zijn right *child*, dat een waarde van 5 retourneert. Bij D wordt $\alpha = \max(3, 5)$, wat 5 is. De waarde van node D is dus 5.

Nog een voorbeeld (tussenstap 1)



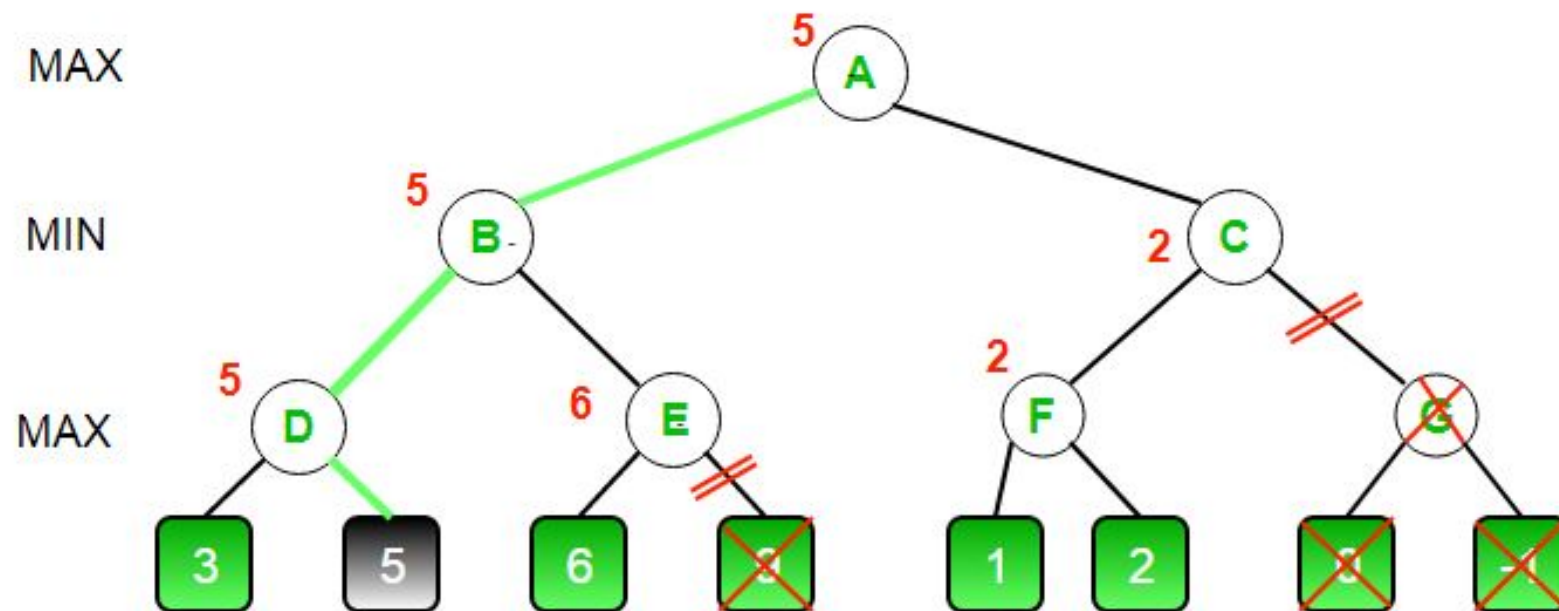
- D retourneert een waarde van 5 naar B. Bij B wordt $\beta = \min(+\infty, 5)$, wat 5 is. De minimizer heeft nu gegarandeerd een waarde van 5 of lager. B roept nu E aan om te zien of hij een lagere waarde dan 5 kan krijgen.
- Bij E zijn de waarden van α en β niet $-\infty$ en $+\infty$, maar respectievelijk $-\infty$ en 5, omdat de waarde van β bij B werd aangepast en B die waarde doorgaf aan E.
- E kijkt nu naar zijn left *child*, die de waarde 6 heeft. Bij E wordt $\alpha = \max(-\infty, 6)$, wat 6 is. Hier wordt de conditie $\beta \leq \alpha$ waar. β is 5 en α is 6. Dus $\beta \leq \alpha$ is waar. Daarom stopt E met zoeken en retourneert 6 aan B.
- Merk op dat het niet uitmaakte wat de waarde van E's right *child* zou zijn. Het had $+\infty$ of $-\infty$ kunnen zijn; het maakte niets uit. We hoefden er nooit naar te kijken omdat de minimizer een waarde van 5 of lager gegarandeerd had. Dus zodra de maximizer de 6 zag, wist hij dat de minimizer deze weg nooit zou kiezen omdat hij links bij B al een 5 kon krijgen. Op deze manier hoefden we niet naar de 9 te kijken en werd computationele tijd bespaard.
- E retourneert een waarde van 6 naar B. Bij B wordt $\beta = \min(5, 6)$, wat 5 is. De waarde van node B is dus ook 5.

Nog een voorbeeld (tussenstap 2)



- B retourneert 5 aan A. Bij A wordt $\alpha = \max(-\text{INF}, 5)$, wat 5 is. Nu is de maximizer gegarandeerd van een waarde van 5 of hoger. A roept nu C aan om te zien of het een hogere waarde dan 5 kan krijgen.
Bij C zijn $\alpha = 5$ en $\beta = +\text{INF}$. C roept F aan.
- Bij F zijn $\alpha = 5$ en $\beta = +\text{INF}$. F kijkt naar zijn left *child*, dat een waarde van 1 heeft. $\alpha = \max(5, 1)$, wat nog steeds 5 is.
F kijkt naar zijn right *child*, dat een waarde van 2 heeft. De beste waarde van deze node is dus 2. α blijft 5.
- F retourneert een waarde van 2 aan C. Bij C wordt $\beta = \min(+\text{INF}, 2)$. De conditie $\beta \leq \alpha$ wordt nu waar omdat $\beta = 2$ en $\alpha = 5$. Dus het proces wordt afgebroken en de hele sub-tree van G hoeft niet eens meer berekend te worden.

Tussenstap 3 (einde)



- De intuïtie achter dit afbreken is dat de minimizer bij C gegarandeerd was van een waarde van 2 of lager. Maar de maximizer was al gegarandeerd van een waarde van 5 als hij voor B zou kiezen. Waarom zou de maximizer C ooit kiezen om een waarde lager dan 2 te krijgen? Opnieuw kun je zien dat het niet uitmaakte wat die laatste twee waarden waren. We besparen bovendien veel rekentijd door een hele sub-tree over te slaan.
- C retourneert nu een waarde van 2 aan A. Daarom is de beste waarde bij A: $\max(5, 2)$, wat 5 is. De optimale waarde die de maximizer kan krijgen is dus 5.

Alpha beta pruning

- Parameters meegeven in de minimax

```
def minimax(self, depth, is_maximizing, alpha, beta):  
    #maximizing  
    best = max(best, new_move.minimax(depth + 1, not is_maximizing,  
alpha, beta)) #tegengestelde positie  
    alpha = max(alpha, best)  
    if beta <= alpha:  
        break # alpha pruning  
  
    # minimizing  
    best = min(best, new_move.minimax(depth + 1, not is_maximizing,  
alpha, beta))  
    beta = min(beta, best)  
    if beta < alpha:  
        break: # beta pruning
```

Alpha beta pruning

- Snijdt stukken weg **zonder de uiteindelijke uitkomst van minimax te wijzigen**
- Dit is dus **altijd efficiënt** om te implementeren!
- In tegenstelling tot depth-limited oplossingen, blijft alpha-beta pruning een exacte oplossing garanderen
- Uiteraard kan je alpha beta pruning én depth-limited **combineren**

Wat met stochastische spellen ?

Minimax is pessimistisch

- Minimax gaat ervan uit dat de tegenstrever perfect speelt
- Dit is niet altijd zo
 - Hetzij door menselijke fouten
 - Hetzij omdat er een fundamentele kanscomponent is, zoals gooien met een dobbelsteen
- Het is dan niet meer zo slim om altijd pessimistisch te redeneren zoals minimax
- We kunnen ook kanstheoretisch beginnen redeneren, en dan bekomen we **expectimax**

Notatie in diagrammen

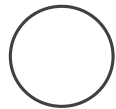
- Om dingen overzichtelijk te houden:



- Driehoek boven voor Max

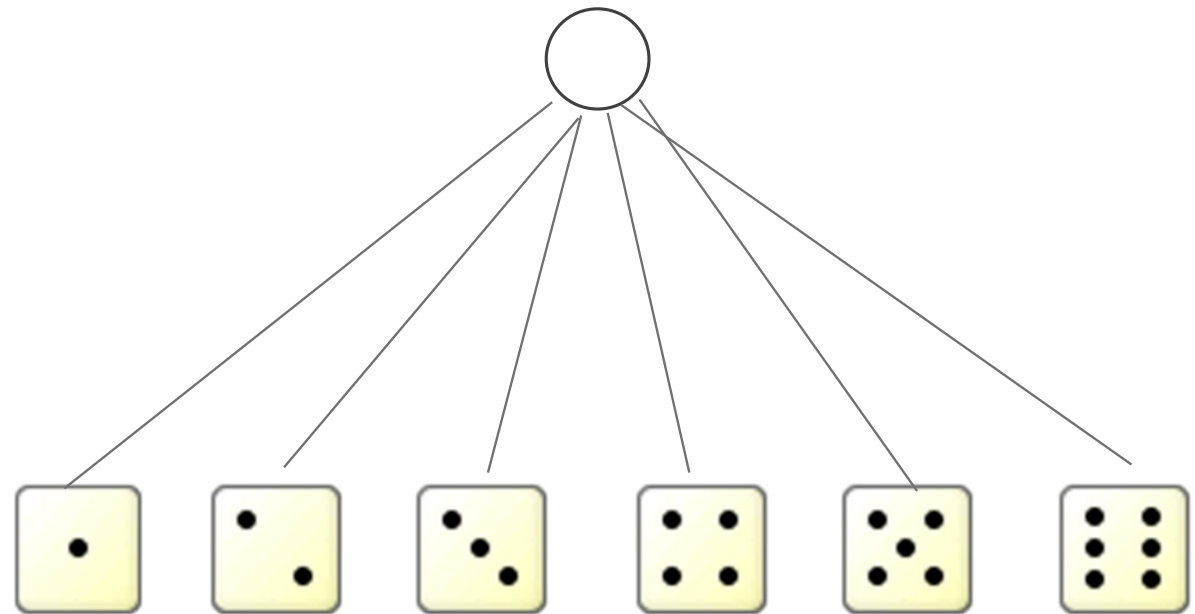


- Driehoek onder voor Min



- Cirkel voor kans

- Een proces van dobbelstenen gooien is dus:

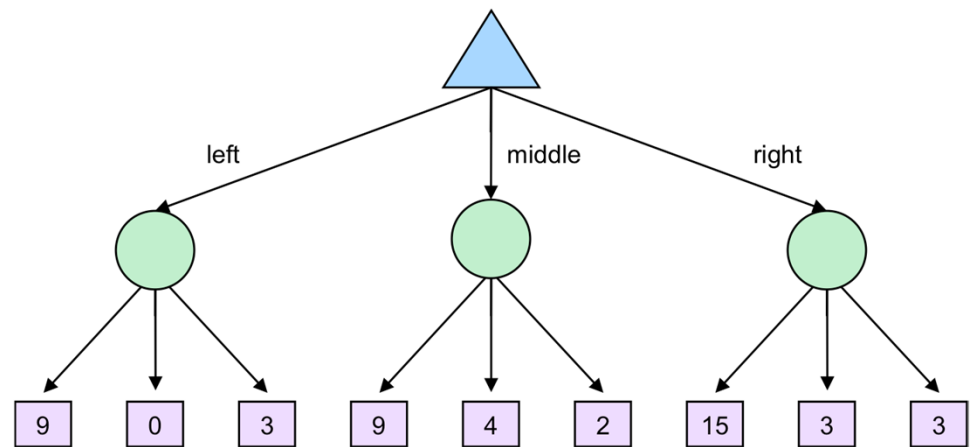


Oorzaken van een kanstheoretisch proces

- Onderliggende **kansactie**
 - Dobbelsteen gooien,
 - Kaart trekken,
 - Random initialisatie bij computerspel,
 - ...
- Dus bij een stochastisch = niet-deterministische situatie
- Indien geen zicht op state van tegenspeler (**imperfect information**)
 - Poker
 - Solitaire
 - 21'en
- Je kan je vermoedens over de opties van de tegenstrever in dit model gieten

Expectimax

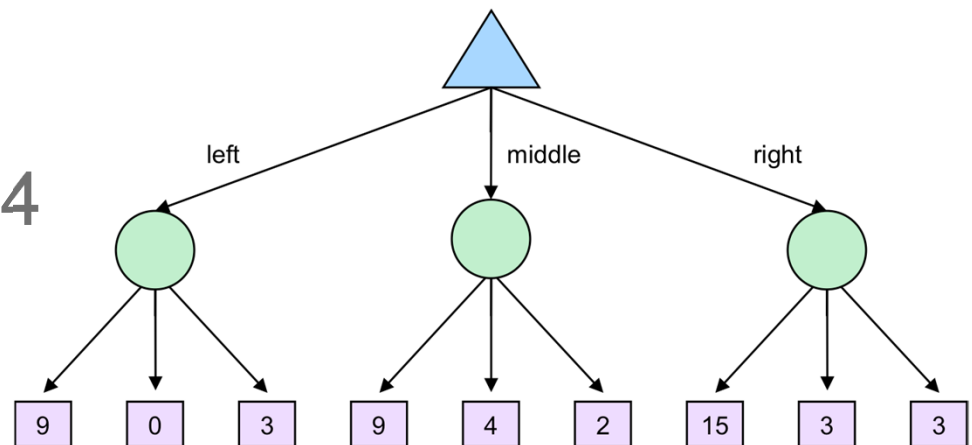
- Expectimax is hetzelfde als minimax, maar in plaats van een minimum/maximum te nemen, neem je voor de kansopties de verwachte waarde (*expected value*) gebaseerd op de kansverdeling.



Expectimax

- Indien alle opties hieronder evenveel kans hebben, is dat dus links

$$9 * 1/3 + 0 * 1/3 + 3 * 1/3 = 4$$



Verschillen minimax, expectimax

- Minimax gaat uit van een ideale tegenstrever, die exact weet wat hij/zij doet
- Minimax is typisch voor zero-sum games
- Perfect information games
- De evaluatiefunctie is statisch
- Expectimax is voor spellen met onzekerheid
- Of voor tegenstrevers waarvan geweten is dat ze een bepaald percentage fouten maken
- Inperfect, stochastic games
- om game states te schatten is er naast een evaluatiefunctie ook een kansverdeling nodig voor elke mogelijke set acties